

Modeling Real Travel Routes Using Shortest-Path Algorithms

Jennifer M.

February 2026

1 Abstract

Shortest-path algorithms are a fundamental topic in graph theory and are widely used to model routing problems in transportation networks. In these models, road systems are represented as weighted graphs, and routes are chosen to minimize quantities such as distance or travel time. While these algorithms are efficient and mathematically understood, the usefulness depends on how well their assumptions reflect real travel conditions. This paper examines how classical shortest-path algorithms are applied to transportation networks and discusses the limitations of these models, specifically the assumption that travel costs are fixed.

2 Introduction

Graphs provide a natural way to represent networks, including transportation systems. Locations such as intersections can be modeled as vertices, while roads connecting them are represented as edges. By assigning weights to edges, additional information such as distance or expected travel time can be incorporated.

One of the most important problems in graph theory is the shortest-path problem, which involves finding the minimum-cost path between two vertices. Classic algorithms for solving this problem, such as Dijkstra's algorithm, are well known and widely taught in discrete mathematics and computer science courses [1] [4].

Although shortest-path algorithms are frequently used in routing applications, they rely on simplifying assumptions that may not hold in real transportation networks. Travel times on roads often vary due to congestion, traffic signals, or time-of-day effects. As a result, routes computed using fixed edge weights may not accurately reflect real travel behavior. This paper explores how shortest-path algorithms are used to model travel routes and examines the limitations of these models when applied to real-world transportation systems.

3 Mathematical Background

A graph is defined as an ordered pair $G=(V,E)$, where V is a set of vertices and E is a set of edges connecting pairs of vertices. In a weighted graph, each edge e exists in E is assigned a real-valued weight $w(e)$, representing the cost of traversing that edge.

In transportation models, vertices often correspond to physical locations, while edges represent direct travel connections such as roads. The weight of an edge may represent distance, average travel time, or another measure of cost.

The shortest-path problem consists of finding a path between two vertices that minimizes the sum of the weights along that path. Dijkstra's algorithm is a standard method for solving this problem in graphs with nonnegative edge weights [3]. The algorithm assumes that all edge weights are fixed and known in advance, and under these conditions it is guaranteed to find an optimal path.

4 Shortest Path Algorithms in Transportation Modeling

Shortest-path algorithms are commonly used in routing and navigation systems. In these applications, transportation networks are modeled as graphs, and routes are selected based on criteria such as minimizing distance or travel time [1].

In practice, edge weights are often estimated using average or typical travel conditions. This approach can produce reasonable routes when traffic patterns are stable or when exact accuracy is not required. For example, in areas with low congestion or during off-peak hours, static shortest-path models may closely approximate actual travel times.

However, this modeling approach simplifies the complexity of real transportation systems. Travel conditions can change significantly due to congestion, accidents, or traffic controls, which are not captured by fixed edge weights. As a result, shortest-path algorithms may produce routes that are optimal according to the model but suboptimal in practice.

5 Limitations of Classical Shortest-Path Models

A major limitation of classical shortest-path models is the assumption that edge weights remain constant. In real transportation networks, travel times often vary depending on the time of day or current traffic conditions. A route that minimizes travel time under one set of conditions may no longer be optimal when conditions change.

Another limitation is that shortest-path algorithms typically optimize a single criterion, such as distance or time. Real travelers may consider additional factors, including route familiarity, toll costs, or personal preferences. These factors are difficult to represent within a simple weighted graph model.

Furthermore, classical models assume complete and accurate information about the network. Routing decisions are often made using estimates or predictions, which introduces uncertainty into the model.

6 Time-Dependent Modeling

One way to address the limitations of static shortest-path models is to allow edge weights to vary over time. In such time-dependent models, the cost of traversing an edge depends on factors such as the time of day or current traffic conditions. This allows predictable changes, such as rush-hour congestion, to be incorporated into routing decisions.

While time-dependent models can better reflect real travel conditions, they are more complex than static models and may require additional assumptions to ensure that shortest paths are well defined [2]. As a result, these models are often used selectively, depending on the level of accuracy required.

Refer to Appendix A. This example shows that shortest-path algorithms select routes based on the total weight of the edges. When travel times are increased to represent congestion, the selected route changes. This highlights how strongly routing decisions depend on the assumptions used to define edge weights.

7 Conclusion

Shortest-path algorithms offer a clear and effective mathematical framework for modeling routing problems in transportation networks. Their simplicity and efficiency make them useful in many practical applications. However, their accuracy depends strongly on the assumptions used to represent travel costs.

Classical shortest-path models with fixed edge weights provide a useful starting point but may fail to capture important features of real travel behavior. Allowing travel costs to change over time can improve realism, but this comes at the cost of increased complexity. This illustrates the trade-off between mathematical simplicity and realism that arises when applying abstract mathematical models to real-world systems.

References

- [1] Thomas H Cormen, Charles Eric Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to algorithms*. The Mit Press, Cambridge, Massachusetts, 2022.
- [2] Brian Dean. *Shortest Paths in FIFO Time-Dependent Networks: Theory and Algorithms*.
- [3] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, Dec 1959.

- [4] Jon Kleinberg and Eva Tardos. *Algorithm design*. Pearson/Addison-Wesley, Boston, 2006.

A Python Code for Shortest-Path Simulation

Listing 1: Python code for modeling real travel routes using shortest-path logic

```
# Code for Math 2030 Module Two Paper on Modeling Real Travel
# Routes Using Shortest-Path Algorithms

# Define a simple road network dictionary
# Each location maps to its neighbors with travel times

graph = {
    "A": {"B": 5, "C": 12},
    "B": {"A": 5, "C": 4},
    "C": {"A": 12, "B": 4}
}

# Travel times with possible routes
route_1 = ["A", "B", "C"]
route_2 = ["A", "C"]

time_1 = graph["A"]["B"] + graph["B"]["C"]
time_2 = graph["A"]["C"]

# Comparison of routes
if time_1 < time_2:
    shortest_path = route_1
    shortest_time = time_1
else:
    shortest_path = route_2
    shortest_time = time_2

print("Shortest Route:", shortest_path)
print("Total Travel Time:", shortest_time)

# Create a simulation of congestion that increases travel time
graph["B"]["C"] = 15

# Recalculate route times
time_1 = graph["A"]["B"] + graph["B"]["C"]
time_2 = graph["A"]["C"]

# Compare routes again
```

```
if time_1 < time_2:
    shortest_path = route_1
    shortest_time = time_1
else:
    shortest_path = route_2
    shortest_time = time_2

print("Shortest Route with Congestion:", shortest_path)
print("Total Travel Time with Congestion:", shortest_time)
```